Lab 5: Optimal Dike Height

CEVE 421/521

2026-02-20

1 Overview

In Lab 4, you ran ICOW to analyze flood risk for a city with a fixed dike. Today you'll flip the question: **how tall should the dike be?**

A taller dike costs more to build but reduces expected flood damage. We'll use optimization to find the height that minimizes total cost, then see how the answer shifts under different sea-level rise assumptions.

References:

- ICOW.jl documentation — the flood risk model
- SimOptDecisions documentation — the optimization framework

2 Before Lab

Complete these steps BEFORE coming to lab:

1. **Accept the GitHub Classroom assignment** (link on Canvas)
2. **Clone your repository:**

```
git clone https://github.com/CEVE-421-521/lab-05-S26-yourusername.git
cd lab-05-S26-yourusername
```

3. **Open the notebook** in VS Code or Quarto preview
4. **Run the first code cell** — it will automatically install any missing packages (this may take 10-15 minutes the first time)

> ! Important
>
> **Submission:** You will render this notebook to PDF and submit the PDF to Canvas. See Section 9 for details.

> ⚠ Warning
>
> **This lab is computationally heavy.** The optimization cells can take a minute or more to run. Rather than using Quarto preview (which re-renders every time you save), we recommend running cells one at a time in VS Code using `Shift+Enter`. Save `quarto render` for the end when you're ready to produce your PDF.

3 Setup

3.1 Load Packages

```
# install SimOptDecisions and ICOW
try
    using SimOptDecisions
    using ICOW
catch
    import Pkg
    try
        Pkg.rm("SimOptDecisions")
    catch
    end
    try
        Pkg.rm("ICOW")
    catch
    end
    Pkg.add(; url="https://github.com/dossgollin-lab/SimOptDecisions")
    Pkg.add(; url="https://github.com/dossgollin-lab/ICOW.jl")
    Pkg.instantiate()
    using SimOptDecisions
    using ICOW
end
using ICOW.EAD
```

```
using CairoMakie
using DataFrames
using Metaheuristics
using NetCDF
using Random
using Statistics

include("sea_level_data.jl")

Random.seed!(2026)
```

3.2 Storm Surge Parameters

We model the annual maximum storm surge using a Generalized Extreme Value (GEV) distribution, the same one from Lab 4. The GEV CDF is

$$F(x) = \exp\left\{-\left[1 + \xi\left(\frac{x-\mu}{\sigma}\right)\right]^{-1/\xi}\right\}$$

where the three parameters control different aspects of the distribution:

```
μ = 3.0   # GEV location (m) — typical annual max surge
σ = 1.0   # GEV scale (m) — spread of annual maxima
ξ = 0.15  # GEV shape — positive means heavy-tailed
```

Line 1

μ (location): centers the distribution — sets the typical annual maximum surge

Line 2

σ (scale): controls the spread — larger means more year-to-year variability

Line 3

ξ (shape): controls the tail — positive means rare surges can be *much* larger than the typical one (a "heavy tail")

4 Set Up the Optimization

In lecture you saw the XLRM framework: e**X**ogenous uncertainties, **L**evers, **R**elationships (the model), and **M**etrics. This lab puts all four pieces together computationally using two packages:

- **ICOW** is the **R**elationship model — it computes flood damage given a dike height, surge hazard, and sea-level rise trajectory
- **SimOptDecisions** is the optimization framework — it searches over **L**evers to minimize a **M**etric, given the e**X**ogenous uncertainties

4.1 The Decision: Dike Height

ICOW's built-in `StaticPolicy` has five defense levers (withdrawal, dike height, dike base, resistance, flood-proofing), but we only want to vary one. Rather than using all five and trying to hold four constant, we define our own policy type with a single decision variable.

`@policydef` from SimOptDecisions lets us create custom policy types. Each `@continuous` field becomes a parameter the optimizer can search over:

```
SimOptDecisions.@policydef DikeOnlyPolicy begin
    @continuous a_frac 0.0 0.8
end
```

Line 2

Declares `a_frac` as a continuous decision variable that the optimizer can vary between 0.0 and 0.8

The variable `a_frac` represents the fraction of city elevation allocated to dike height. With ICOW's default city geometry, the physical dike height is

$$D = 0.8 \times \texttt{a\_frac} \times 17 \quad \text{m}$$

so `a_frac = 0.3` gives roughly a 4 m dike, and `a_frac = 0.8` gives the maximum ~11 m dike.

4.2 Connecting Our Policy to ICOW

ICOW's simulation expects a full `StaticPolicy`, so we need a bridge function that converts our simple one-parameter policy into a full one (with the other levers set to their defaults):

```
function to_static(policy::DikeOnlyPolicy)
    return StaticPolicy(;
        a_frac=SimOptDecisions.value(policy.a_frac),
        w_frac=0.0,   # no withdrawal
        b_frac=0.2,   # reasonable dike base
        r_frac=0.0,   # no resistance
        P=0.0,        # no flood-proofing
    )
end
```

Next, we teach SimOptDecisions how to simulate our custom policy type. This uses Julia's *multiple dispatch*: we add a new method to `SimOptDecisions.simulate` that accepts our `DikeOnlyPolicy`, converts it, and forwards to ICOW.

```
function SimOptDecisions.simulate(
    config::EADConfig, scenario::EADScenario, policy::DikeOnlyPolicy,
rng::AbstractRNG
)
    return SimOptDecisions.simulate(config, scenario, to_static(policy), rng)
end
```

4.3 Scenario: Surge + Sea-Level Rise

An `EADScenario` bundles everything the decision-maker can't control: the surge hazard (GEV parameters), a sea-level rise trajectory, and a discount rate. ICOW uses these to compute expected annual damage at each year, then discounts future costs back to present value.

First, load the sea-level rise projections from the BRICK model:

```
brick_data = load_brick_projections()
rcp85 = get_brick_scenario(brick_data, "rcp85")
rcp85_quantiles = brick_quantiles(rcp85, [0.5])
n_years = 50
slr_rcp85_median = rcp85_quantiles[1:n_years, 1]
```

Now build the scenario. The discount rate r determines how much we value future damages relative to present costs — a dollar of damage t years from now has a present value of $\frac{1}{(1+r)^t}$. At $r = 0.03$, damage 50 years out is worth only about \$0.22 today.

```
config = EADConfig(; n_years=n_years)
scenario_rcp85 = EADScenario(;
    surge_loc=μ,
    surge_scale=σ,
```

```
    surge_shape=ξ,
    mean_sea_level=slr_rcp85_median,
    discount_rate=0.03,
)
```

Line 1

50-year planning horizon with default city geometry

Line 3

GEV parameters for the storm surge distribution (same as Lab 4)

Line 6

SLR trajectory — one value per year from the BRICK median projection

Line 7

3% annual discount rate for computing net present value

4.4 Objective: Minimize Total Cost

The total cost has two competing components:

$$\text{Total Cost } (h) = \text{Investment } (h) + \text{NPV } \left[\text{ Expected Damage } (h) \right]$$

where the net present value of expected damage over the planning horizon is

$$\text{NPV} = \sum_{t=1}^{T} \frac{E[\text{Damage}_t(h)]}{(1+r)^t}$$

Investment increases with dike height h, while expected damage decreases. This is the benefit-cost framework from lecture: we want the dike height where the marginal cost of building higher equals the marginal reduction in expected damage.

The `calculate_metrics` function receives one outcome per scenario, sums investment + damage for each, and averages across scenarios. With a single scenario the average is trivial, but this matters when we optimize over an ensemble later.

```
function calculate_metrics(outcomes)
    total_costs = [
        SimOptDecisions.value(o.investment) +
SimOptDecisions.value(o.expected_damage)
        for o in outcomes
    ]
    return (; total_cost=mean(total_costs))
end
```

Line 3

For each scenario outcome, sum the two cost components: upfront investment and discounted expected flood damage

Line 6

Average across all scenarios — with one scenario this is just the total cost; with an ensemble it becomes the expected cost

4.5 Sanity Check

Before letting an optimizer loose, it's good practice to evaluate a few policies by hand. `evaluate_policy` runs the ICOW model for a given policy and scenario — the same EAD calculation you saw in Lab 4, but now repeated for different dike heights.

```
let
    results = map([0.0, 0.1, 0.2, 0.3, 0.4, 0.5]) do a
        policy = DikeOnlyPolicy(; a_frac=a)
        fd = FloodDefenses(to_static(policy), config)
        metrics = evaluate_policy(
            config, [scenario_rcp85], policy, calculate_metrics
        )
        (
            a_frac=a,
            dike_height_m=round(fd.D; digits=1),
            total_cost_B=round(metrics.total_cost / 1e9; digits=2),
        )
    end
    DataFrame(results)
end
```

Table 1: Total cost for different dike heights under RCP 8.5 median SLR

Row	a_frac	dike_height_m	total_cost_B
	Float64	Float64	Float64
1	0.0	0.0	687.31
2	0.1	1.4	650.52
3	0.2	2.7	487.88
4	0.3	4.1	358.57
5	0.4	5.4	331.74
6	0.5	6.8	345.34

Question 1

Look at Table 1. The total cost first decreases, then increases as `a_frac` grows. What two forces create this U-shaped cost curve? Why is neither `a_frac = 0` nor `a_frac = 0.5` optimal?

5 Find the Optimal Dike Height

5.1 How `SimOptDecisions.optimize()` Works

The sanity check showed a U-shaped cost curve — there's a minimum somewhere in the middle. Instead of testing every value by hand, we use `SimOptDecisions.optimize()` to search automatically. Here's what it does on each iteration:

1. Propose a candidate `a_frac` value
2. Build a `DikeOnlyPolicy` from it
3. Run ICOW via `simulate()` for that policy against each scenario
4. Pass the results to `calculate_metrics` (sum investment + damage, average across scenarios)
5. Use the metric to decide where to search next

We use ECA (Evolutionary Centers Algorithm), a population-based metaheuristic that maintains multiple candidate solutions and iteratively improves them.

5.2 Single-Scenario Optimization

First, configure the optimization backend. A `MetaheuristicsBackend` wraps an algorithm from Metaheuristics.jl.

```
backend = MetaheuristicsBackend(;
    algorithm=:ECA,          # Evolutionary Centers Algorithm
    max_iterations=50,       # how many rounds of improvement
    population_size=20,      # candidates evaluated per round
)
```

Now run the optimization. The search bounds come from the `@policydef` definition (`a_frac ∈ [0, 0.8]`), so we don't need to specify them separately.

```
result_rcp85 = SimOptDecisions.optimize(
    config,
    [scenario_rcp85],
    DikeOnlyPolicy,
    calculate_metrics,
    [minimize(:total_cost)];
    backend=backend,
)
```

Line 2

`EADConfig` — city geometry and planning horizon

Line 3

List of scenarios to evaluate against (just one here)

Line 4

Policy type the optimizer will search over

Line 5

Function that computes the objective from simulation outcomes

Line 6

Minimize the `total_cost` metric returned by `calculate_metrics`

Line 7

Which optimization algorithm and how hard to search

5.3 Inspect the Result

The result contains `pareto_params` (best parameters found) and `pareto_objectives` (corresponding cost).

```
optimal_a_rcp85 = result_rcp85.pareto_params[1][1]       # best a_frac found
optimal_cost_rcp85 = result_rcp85.pareto_objectives[1][1] # corresponding total cost

fd_rcp85 = FloodDefenses(to_static(DikeOnlyPolicy(; a_frac=optimal_a_rcp85)), config)

println("Optimal a_frac: $(round(optimal_a_rcp85; digits=3))")
println("Optimal dike height: $(round(fd_rcp85.D; digits=2)) m")
println("Minimum total cost: \$$(round(optimal_cost_rcp85 / 1e9; digits=2))B")
```

```
Optimal a_frac: 0.404
Optimal dike height: 5.49 m
Minimum total cost: $331.72B
```

5.4 A Different Scenario

That result assumed RCP 8.5 (high emissions). Let's re-optimize under RCP 2.6 (aggressive emissions cuts) and see how much the answer changes.

```
rcp26 = get_brick_scenario(brick_data, "rcp26")
rcp26_quantiles = brick_quantiles(rcp26, [0.5])
slr_rcp26_median = rcp26_quantiles[1:n_years, 1]

scenario_rcp26 = EADScenario(;
    surge_loc=μ,
    surge_scale=σ,
    surge_shape=ξ,
    mean_sea_level=slr_rcp26_median,
    discount_rate=0.03,
)
```

Same `optimize()` call, just with a different scenario:

```
result_rcp26 = SimOptDecisions.optimize(
    config,
    [scenario_rcp26],
    DikeOnlyPolicy,
    calculate_metrics,
    [minimize(:total_cost)];
```

```
    backend=backend,
)
```

5.5 Compare

```
let
    optimal_a_rcp26 = result_rcp26.pareto_params[1][1]
    optimal_cost_rcp26 = result_rcp26.pareto_objectives[1][1]
    fd_rcp26 = FloodDefenses(
        to_static(DikeOnlyPolicy(; a_frac=optimal_a_rcp26)), config
    )

    DataFrame(
        Scenario=["RCP 8.5 (median SLR)", "RCP 2.6 (median SLR)"],
        Optimal_a_frac=[round(optimal_a_rcp85; digits=3), round(optimal_a_rcp26;
digits=3)],
        Dike_Height_m=[round(fd_rcp85.D; digits=2), round(fd_rcp26.D; digits=2)],
        Total_Cost_B=[
            "\$$(round(optimal_cost_rcp85 / 1e9; digits=2))B",
            "\$$(round(optimal_cost_rcp26 / 1e9; digits=2))B",
        ],
    )
end
```

Table 2: Optimal dike height under two different SLR scenarios

Row	Scenario	Optimal_a_frac	Dike_Height_m	Total_Cost_B
	String	Float64	Float64	String
1	RCP 8.5 (median SLR)	0.404	5.49	$331.72B
2	RCP 2.6 (median SLR)	0.401	5.45	$323.99B

5.6 Visualize the Cost Curves

```
let
    a_values = 0.0:0.01:0.6
    costs_rcp85 = map(a_values) do a
        metrics = evaluate_policy(config, [scenario_rcp85], DikeOnlyPolicy(;
a_frac=a), calculate_metrics)
        metrics.total_cost / 1e9
    end
    costs_rcp26 = map(a_values) do a
        metrics = evaluate_policy(config, [scenario_rcp26], DikeOnlyPolicy(;
a_frac=a), calculate_metrics)
        metrics.total_cost / 1e9
    end

    fig = Figure(; size=(700, 400))
    ax = Axis(
        fig[1, 1];
```

```
        xlabel="a_frac (dike investment fraction)",
        ylabel="Total Cost (\$B)",
        title="Cost Curves Under Two SLR Scenarios",
    )

    lines!(ax, collect(a_values), costs_rcp85; color=:red, linewidth=2, label="RCP
8.5 median")
    lines!(ax, collect(a_values), costs_rcp26; color=:blue, linewidth=2, label="RCP
2.6 median")

    scatter!(
        ax,
        [optimal_a_rcp85],
        [optimal_cost_rcp85 / 1e9];
        color=:red,
        markersize=15,
        marker=:star5,
    )
    optimal_a_rcp26 = result_rcp26.pareto_params[1][1]
    optimal_cost_rcp26 = result_rcp26.pareto_objectives[1][1]
    scatter!(
        ax,
        [optimal_a_rcp26],
        [optimal_cost_rcp26 / 1e9];
        color=:blue,
        markersize=15,
        marker=:star5,
    )

    axislegend(ax; position=:rt)
    fig
end
```

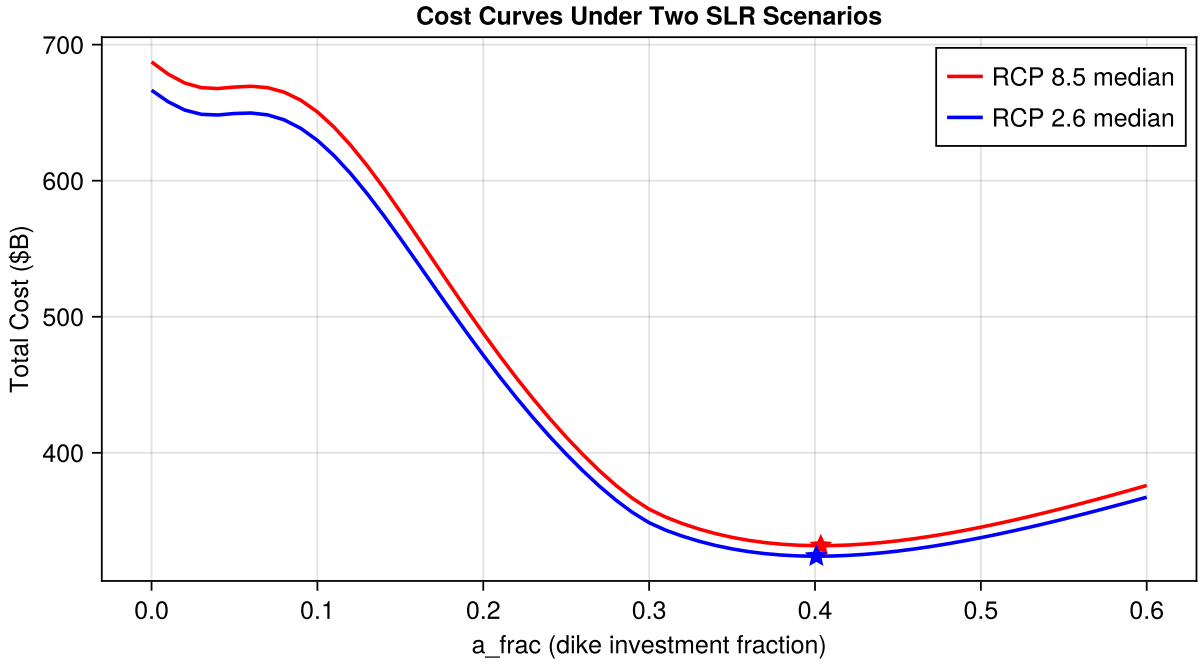

Figure 1: Total cost vs. dike height under two SLR scenarios. Stars mark the optima.

Question 2

Look at Table 2 and Figure 1.

1. *How different are the optimal dike heights under RCP 8.5 vs. RCP 2.6? Are you surprised by how similar (or different) they are?*
2. *Our planning horizon is 50 years and our discount rate is 3%. How might each of these choices contribute to the similarity? (Hint: RCP 8.5 and 2.6 diverge most after 2060-2070, and a 3% discount rate means a dollar of damage 50 years from now is worth only about $0.22 today.)*
3. *What changes to the problem setup — time horizon, discount rate, or something else — might make the two scenarios diverge more?*

6 Optimize Over an Ensemble

The two median scenarios gave similar answers — but medians compress the full range of uncertainty into a single number. BRICK actually gives us many plausible SLR trajectories, some much more extreme than the median. What if we optimize over an *ensemble* of them at once?

Mathematically, we're solving

$$\min_h \frac{1}{N} \sum_{i=1}^{N} \text{Total Cost}_i(h)$$

where i indexes ensemble members. Each member has the same surge distribution but a different SLR trajectory, so this finds the dike height that minimizes *expected* total cost across the ensemble — implicitly treating each member as equally likely.

6.1 Build the Ensemble

Each column of `get_brick_scenario()` is one ensemble member's SLR trajectory. We build a vector of `EADScenario` objects — one per member. The surge parameters stay the same; what varies is the SLR path.

```
N_ensemble = 3
rcp85_full = get_brick_scenario(brick_data, "rcp85")

ensemble_scenarios = [
    EADScenario(;
        surge_loc=μ,
        surge_scale=σ,
        surge_shape=ξ,
        mean_sea_level=rcp85_full[1:n_years, i],
        discount_rate=0.03,
    )
    for i in 1:N_ensemble
]
```

Line 1

Start small so the optimization runs quickly. Try increasing to 30 or 50 and see how the result changes!

Line 9

Column i of the BRICK output gives ensemble member i's sea-level rise trajectory

> **Tip**
>
> **Try it:** After you've run the ensemble optimization once with `N_ensemble = 3`, come back and increase it (e.g., to 10, 30, or 50) to see how the optimal dike height shifts as you include more SLR futures. More members means a better sample of the uncertainty — but each optimization evaluation takes proportionally longer. How would you decide when `N_ensemble` is "large enough"? (Hint: what should happen to the optimal dike height as you keep adding members?)

6.2 Optimize

Same `optimize()` call as before, but now we pass all `N_ensemble` scenarios. Each function evaluation runs ICOW `N_ensemble` times (once per scenario), so each iteration is more expensive. We compensate by using a smaller population — with only one decision variable, a large population wastes effort exploring a 1D space. More iterations are cheap relative to more candidates per iteration, and they give the optimizer time to converge.

```
ensemble_backend = MetaheuristicsBackend(;
    algorithm=:ECA,
    max_iterations=100,
```

```
    population_size=5,
)
```

Line 3

Many iterations to ensure convergence — each iteration is cheap (only `population_size` evaluations)

Line 4

Small population because we only have one decision variable — no need to explore a high-dimensional space

```
result_ensemble = SimOptDecisions.optimize(
    config,
    ensemble_scenarios,
    DikeOnlyPolicy,
    calculate_metrics,
    [minimize(:total_cost)];
    backend=ensemble_backend,
)
```

Line 3

All `N_ensemble` scenarios are passed — the optimizer evaluates every candidate dike height against all of them

Line 5

`calculate_metrics` averages total cost across all outcomes, so the optimizer minimizes the *mean* cost

6.3 Inspect the Result

```
optimal_a_ensemble = result_ensemble.pareto_params[1][1]
optimal_cost_ensemble = result_ensemble.pareto_objectives[1][1]
fd_ensemble = FloodDefenses(to_static(DikeOnlyPolicy(; a_frac=optimal_a_ensemble)),
config)

println("Ensemble-optimal a_frac: $(round(optimal_a_ensemble; digits=3))")
println("Ensemble-optimal dike height: $(round(fd_ensemble.D; digits=2)) m")
println("Minimum expected total cost: \$$(round(optimal_cost_ensemble / 1e9;
digits=2))B")
```

```
Ensemble-optimal a_frac: 0.403
Ensemble-optimal dike height: 5.49 m
Minimum expected total cost: $332.02B
```

6.4 Compare All Results

```
let
    optimal_a_rcp26 = result_rcp26.pareto_params[1][1]
    optimal_cost_rcp26 = result_rcp26.pareto_objectives[1][1]
    fd_rcp26 = FloodDefenses(
        to_static(DikeOnlyPolicy(; a_frac=optimal_a_rcp26)), config
    )

    DataFrame(
        Scenario=[
            "RCP 8.5 (median only)",
            "RCP 2.6 (median only)",
            "RCP 8.5 ($(N_ensemble)-member ensemble)",
        ],
        Optimal_a_frac=[
            round(optimal_a_rcp85; digits=3),
            round(optimal_a_rcp26; digits=3),
            round(optimal_a_ensemble; digits=3),
        ],
        Dike_Height_m=[
            round(fd_rcp85.D; digits=2),
            round(fd_rcp26.D; digits=2),
            round(fd_ensemble.D; digits=2),
        ],
        Total_Cost_B=[
            "\$$(round(optimal_cost_rcp85 / 1e9; digits=2))B",
            "\$$(round(optimal_cost_rcp26 / 1e9; digits=2))B",
            "\$$(round(optimal_cost_ensemble / 1e9; digits=2))B",
        ],
    )
end
```

Table 3: Optimal dike height: single-scenario vs. ensemble optimization

Row	Scenario	Optimal_a_frac	Dike_Height_m	Total_Cost_B
	String	Float64	Float64	String
1	RCP 8.5 (median only)	0.404	5.49	$331.72B
2	RCP 2.6 (median only)	0.401	5.45	$323.99B
3	RCP 8.5 (3-member ensemble)	0.403	5.49	$332.02B

Question 3

Look at Table 3. Compare the ensemble-optimal dike height to the RCP 8.5 and RCP 2.6 single-scenario optima.

1. *Where does the ensemble result fall relative to the two single-scenario optima, and why?*
2. *The ensemble approach implicitly assumes each SLR trajectory in the ensemble is equally likely. Is that a reasonable assumption? What would change if some trajectories were weighted more heavily?*

7 Reflection

Question 4

We found that the "optimal" dike height depends on our assumptions about the future. If you were advising a city council, would you recommend optimizing for one scenario or an ensemble? What are the limitations of either approach, and why might we want something beyond optimization?

Extensions (Optional)

If you finish early, try one of these:

- **Uncertain discount rate:** Re-optimize with discount rates of 0.01, 0.03, and 0.07. Which parameter — SLR or discount rate — has more influence on the optimal dike height?
- **Multi-objective:** Minimize investment and expected damage separately using `[minimize(:investment), minimize(:expected_damage)]`. What does the Pareto front look like?

8 Summary

Key takeaways:

1. **Optimization balances investment against expected damage** — finding the dike height where marginal cost equals marginal benefit
2. **The answer depends on framing** — time horizon, discount rate, and scenario choice all shape the "optimal"
3. **Ensemble optimization hedges across futures** — but the result depends on which futures you include and how you weight them
4. **Always verify your optimizer** — metaheuristics can fail to converge, especially with expensive evaluations

9 Submission

1. Write your answers in the response boxes above
2. **Render to PDF:**
   - In VS Code: Open the command palette (`Cmd+Shift+P` / `Ctrl+Shift+P`) → "Quarto: Render Document" → select Typst PDF
   - Or from the terminal: `quarto render index.qmd --to typst`

3. **Submit the PDF** to the Lab 5 assignment on Canvas
4. **Push your code** to GitHub (for backup):

```
git add -A && git commit -m "Lab 5 complete" && git push
```

Checklist

Before submitting:

- ☐ Packages load without error
- ☐ Sanity check table computed (Table 1)
- ☐ Question 1 answered (U-shaped cost curve)
- ☐ Single-scenario optimization completed for RCP 8.5 and RCP 2.6
- ☐ Scenario comparison table (Table 2) and cost curves (Figure 1) generated
- ☐ Question 2 answered (scenario similarity and what drives it)
- ☐ Ensemble results reviewed (Table 3)
- ☐ Question 3 answered (ensemble vs. single-scenario comparison)
- ☐ Question 4 answered (reflection)
- ☐ Notebook renders to PDF without errors

Bibliography